

legacy-reverse 利用マニュアル

目次

1. skill の配置	4
2. hook の登録（必須）	4
3. MCP サーバの登録（推奨）	4
4. ツール類	4
5. 開始	4
①②の承認とは何をすることか	5
⑤で fail したときの裁定	7
WBS は自動でダッシュボードになる	10
中断しても 1 からやり直しにならない	11
WBS サイト	12
あなたが直接書いてよいファイル	12

レガシーコード（Fortran / C# など）を Python へ仕様ベースで移植するための、Claude Code skill 群です。関数を 1 つずつ、次の流れで進めます。

①リポジトリ解析 → ①関数仕様書 → ②テスト仕様書 → ③テストコード → ④実装 → ⑤テスト
→ ⑥完了検証 → ⑦分析・改善

設計の柱は 5 つあります。

1. **クリーンルーム分業** — ②③はレガシー原文を見ない。③は「②+規約」だけ、④は「①+規約」だけを入力に作られる。独立に作ったテストと実装を⑤で突き合わせることで、仕様書の穴が機械的に炙り出される。
2. **ハッシュ連鎖** — ①→②→③の各成果物は上流のハッシュを記録している。上流が改訂されると下流が自動的に「要再生成 (stale△)」になり、伝搬漏れが起きない。
3. **人の承認ゲート** — 仕様の確定・テスト仕様の承認・裁定は必ず人が行う。AI は「仮説+Yes/No の問い」の形で判断材料を出し、勝手に確定しない。
4. **機械が正、AI は意味づけ** — 関数の列挙 (①) は静的解析プログラムが行い、AI の成果物 (②③) は人に届く前に機械レビュー（根拠引用の实在検証・省略検知・トレーサビリティ突合）を通る。「もっともらしいが根拠のない記述」はあなたの目に触れる前に機械が落とす（詳細は「品質ゲート」の章）。
5. **挙動保存の改善 (⑦)** — 移植完了後の高速化・リファクタリングは、③のテスト資産を安全網に「テスト全 pass 維持」を絶対条件として行う。

あなた（人）の仕事は「トリガを引く」「質問に答える」「承認する」「裁定する」の 4 つです。コードや文書を直接書く必要は基本的にありません（書いてもよい場所は後述）。

作業中に手元へ置くコマンド一覧は [QUICKREF.md](#)（1 枚）にあります。本書は背景と操作の意味を説明する「ドキュメント版」です。

pricing-batch リバースエンジニアリング WBS

公開
2026年7月25日

目次

進捗サマリ

Open ISSUES（要人間判断）

関数一覧（推奨着手順）

⑦ 改善イタレーション

進捗サマリ

①spec	②test-spec	③test-code	④impl	⑤test
3/3	3/3	3/3	3/3	3/3

Open ISSUES（要人間判断）

なし 🚩

関数一覧（推奨着手順）

#	関数	依存	①spec	②test-spec	③test-code	④impl	⑤test
1	get_rate	なし	✅	✅	✅	✅	✅
2	round_val	なし	✅	✅	✅	✅	✅
3	calc_price	F-0001, F-0002	✅	✅	✅	✅	✅

⑦ 改善イタレーション

ID	分類	目的	対象	状態	達成
REF-001	refactor	get_rate のフォールバック読みを分離し複雑度を下げる	F-0001	verified	✅

図 1: WBS（進捗のホーム画面）。進捗サマリ・あなたへの質問（Open ISSUES）・関数一覧・⑦改善イタレーションが 1 画面に集約され、各✅から成果物へ移動できる

セットアップ

1. skill の配置

対象プロジェクトに skills リポジトリから次をコピーします。

```
cp -r skills/legacy-reverse <project>/.claude/skills/  
cp -r skills/legacy-reverse/skills/* <project>/.claude/skills/
```

2. hook の登録（必須）

legacy-reverse/hooks/settings-example.json の内容を <project>/.claude/settings.json にマージします。これは④⑤フェーズ中の tests/ 編集を機械的に拒否する安全装置で、「AI がテストを書き換えて通す」事故をプロンプトではなく仕組みで防ぎます。

3. MCP サーバの登録（推奨）

<project>/mcp.json に次を追加すると、台帳操作・テスト実行などの機械処理が 型付きツールになり、許可プロンプトが減って安定します（未登録でも動作は同じ）。

```
{  
  "mcpServers": {  
    "legacy-reverse": {  
      "command": "python",  
      "args": ["<skills リポジトリ>/mcp-servers/legacy-reverse-mcp/server.py"]  
    }  
  }  
}
```

4. ツール類

- **Quarto** (HTML/PDF 出力) : 未導入なら quarto-typst-pdf skill の qtpdf.py install でポータブル導入（管理者権限不要）
- **Chromium 系ブラウザ** (Chrome/Edge 等) : PDF に Mermaid 図を焼き込むのに使います。HTML だけなら不要 (qtpdf.py doctor で有無を確認できます)
- **Python パッケージ**: pip install pytest sphinx sphinx-rtd-theme (⑦では追加で radon ruff bandit pip-audit)

5. 開始

Claude Code で /legacy-reverse を実行すると、セットアップ状態を確認して 次の一手を案内します。新規なら /legacy-0-analyze へ。

フェーズ別ガイドー各フェーズで人がやること

すべてのフェーズは人がスラッシュコマンドで起動します。AI が勝手に次のフェーズへ進むことはありません。

Table 1

フェーズ	コマンド	人がやること
① 解析	<code>/legacy-0-analyze</code>	レガシーの場所・言語・新パッケージ名を答える。型対応表・規約を一緒に確定（Fortran の関数列挙は静的解析プログラムが行い、AI は説明の充填と抽出レポートの確認のみ）
① 仕様書	<code>/legacy-1-spec F-xxxx</code>	レビューして OK を出す（reviewed 化）。●● 項目の質問に答える
② テスト仕様	<code>/legacy-2-testspec F-xxxx</code>	ケース一覧と質問を見て承認（approved 化）。期待値の不明点に答える
③ テストコード	<code>/legacy-3-testcode F-xxxx</code>	基本は見守り（完了時に freeze される）
④ 実装	<code>/legacy-4-impl F-xxxx</code>	基本は見守り。spec-gap ISSUE が来たら①改訂を指示
⑤ テスト	<code>/legacy-5-test F-xxxx</code>	fail 時の裁定（後述）。pass なら WBS で✅を確認
⑥ 完了検証	<code>/legacy-6-check</code>	全関数完了後に 1 回。不備リストが出たら該当フェーズへ差し戻し
⑦ 分析・改善	<code>/legacy-7-analyze</code>	代表ワークロードを教える。施策票を承認する

「次にどの関数をやるべきか」は WBS の推奨着手順（依存の葉から）に従います。迷ったら `/legacy-reverse` に聞けば next を提案します。

①②の承認とは何をすることか

AI は承認を求めるとき、変更点サマリ・Confidence（●確認済/●推測/●仮定）の内訳・未回答の質問一覧をチャットに提示します。あなたは:

- 内容に問題なければ「OK」と返す → AI がフロントマターの status を更新する
- 質問には Yes / No / 修正内容 で答える（AI は必ず仮説を添えて聞いてくるので、白紙から考える必要はない）
- 答えられない質問は「保留」でよい。ISSUE として残り、WBS の最上部に出続ける

関数仕様書: get_rate

概要

割引区分コードから割引率をテーブル検索して返す。テーブル未ロード時は RATES.DAT から読み込む（フォールバック）。

インタフェース

入力

#	名前	レガシー型	新型	説明	Confidence
1	CODE	CHARACTER*1	str	割引区分コード (1文字、完全一致)	

出力

名前	レガシー型	新型	説明	Confidence
RATE	REAL	float (戻り値テーブル第1要素)	割引率。エラー時 0.0	
IERR	INTEGER	int (戻り値テーブル第2要素)	0=正常 1=コード無し 2=ファイル無し	

グローバル状態

名前	読み/書き	説明	Confidence
/RATTBL/ TCODE(10), TRATE(10), NRATE	読み書き	割引率テーブル (最大10件)。フォールバック時に書き込む	

参照外部ファイル

ファイル	読み/書き	用途	Confidence
RATES.DAT	読み	割引率マスタ。1行 = 区分1文字 + 率5桁 ((A1,F5.3) 、例: A0.050)	

呼び出しサブルーチン

名前	func-id	用途
----	---------	----

図 2: ①関数仕様書。機械抽出した IO 表と、機能詳細の各項目に Confidence () とレガシー行番号の根拠が付く。あなたはこれをレビューして OK を出す

ISSUE は必ず「仮説 + Yes/No の問い」の形で来ます。白紙の質問は来ません。

目次

- 概要
- インタフェース
- 機能詳細
- 副作用・例外
- 未確定事項

割引率テーブルの10件打ち切りはバグ互換で維持するか

事象

GETRT のフォールバック読込は RATES.DAT の先頭10件で打ち切り、11件目以降を黙って無視する (legacy/pricing.f:25、配列サイズ10の制約)。マスタが10件を超えた場合、超過分の区分は常に「コード無し(IERR=1)」になる。エラーも警告も出ない。

質問（人への問い）

仮説: これは配列サイズ由来の制約であり意図した仕様ではないが、移植ではバグ互換で10件打ち切りを維持する（現行マスタは10件以内で運用されていると推測。上限拡張は挙動変更になるため⑦以降で検討）。

問い: この理解で正しいですか？（Yes / No / 修正）

回答（人が記入）

- 回答: Yes. バグ互換で10件打ち切りを維持。上限拡張は⑦以降で検討。
- 回答者 / 日付: havealinch / 2026-07-25

反映

反映先	内容
docs/specs/F-0001.md	SPEC-F0001-02 の10件打ち切りを正式仕様として確定

図 3: ISSUE の例。AI の仮説に Yes/No/修正 で答えるだけでよい。回答はドメイン知識集に転記され、同じ質問は二度と来ない

⑤で fail したときの裁定

テストが落ちたとき、原因は 3 種類あります。(a) だけは AI が自走し、(b)(c) はあなたの承認が要ります。

Table 2

分類	意味	あなたの操作
(a) 実装バグ	実装が①を満たしていない	何もしない（AI が④修正→⑤再実行を回す）
(b) テストコードバグ	③が②とズレている	ISSUE の証拠を見て承認 → ③再生成を指示
(c) 仕様が誤り	②①自体が間違い	ISSUE で裁定 → ①改訂を指示（伝搬は自動検知）

(a) のループは 3 回で自動停止し、triage ISSUE が起票されて WBS に ➊ が出ます。このときの再開手順:

- ISSUE を読んで (a)/(b)/(c) を裁定し、回答欄に記入（またはチャットで回答）
- AI が反映 (applied) したら unblock （AI に「F-xxxx をアンブロックして」でよい）
- /legacy-5-test F-xxxx を再トリガ（試行回数は 1 からやり直し）

テスト結果: get_rate

サマリ

目次

[サマリ](#)[ケース別結果](#)[失敗詳細](#)[試行履歴](#)

仕様ケース数	実装済み	実装率	成功	失敗	スキップ	実行時間
6	6	100%	6	0	0	0.03s

ケース別結果

ケースID	内容	分類	実装	結果	時間	詳細
F0001-TC-001	ロード済みテーブルからのヒット	正常系	✓	✓	0.0s	
F0001-TC-002	未ロード時のフォールバック読み込み	外部ファイルI/O / グローバル状態	✓	✓	0.013s	
F0001-TC-003	10件打ち切り (11件目以降は無視)	境界値	✓	✓	0.015s	
F0001-TC-004	コード不一致	異常系	✓	✓	0.0s	
F0001-TC-005	マスタファイル無し	異常系 / 外部ファイルI/O	✓	✓	0.0s	
F0001-TC-006	空ファイル	境界値	✓	✓	0.001s	

失敗詳細

なし

試行履歴

図 4: ⑤テスト結果報告書。実装率（②のケースが③に全部実装されたか）と、ケースごとの内容・分類・結果が 1 枚で分かる。内容列は②の該当ケース定義へのリンク

品質ゲート — AI の誤りを機械で検知する仕組み

AI がハルシネーション（存在しない根拠の引用・仕様の捏造）や手抜き（記入の省略・スタブ実装）をしても、**人のレビューに届く前に機械が検知して差し戻す仕組み**が各フェーズに入っています。あなたが受け取る成果物は、以下をすべて通過済みです。

Table 3

フェーズ	ゲート	機械が検知するもの
①	抽出器内蔵の完全性突合	2 系統のカウント差分（関数の数え漏れ）。結果は <code>data/extract-report.json</code>
①	<code>review_checks.py spec</code>	実在しないレガシー行への引用 、●（確認済）なのに根拠なし、記入の省略（プレースホルダ残存）、必須節の欠落、レガシー原本の変更
②	<code>review_checks.py testspec</code>	● 仕様項目のテストケース漏れ、①に 存在しない仕様 ID の参照（捏造） 、宙に浮いたケース参照、期待値根拠の規定外表記
③	マーカー突合	②のケース ID とテスト関数の過不足
④	<code>check_stubs.py</code>	空実装・NotImplementedError・TODO/FIXME（スタブ化）
⑤	<code>ledger verify</code>	②の陳腐化・freeze 後のテスト改変・裁定待ちの見落とし
⑥	<code>review_checks.py all</code>	上記①②の全関数総点検

したがって**あなたのレビューは「意味が正しいか」に集中できます**。「引用された行は実在するか」「トレーサビリティは揃っているか」の照合作業は不要です。機械で取れないのは「式は書いてあるがレガシーの意図と違う」という意味レベルのずれで、これは①のレビューと⑤の突き合わせ（独立に作ったテスト vs 実装）が受け持ちます。

大規模プロジェクト（数百～2000 関数）と中断・再開

WBS は自動でダッシュボードになる

200 関数を超えると、WBS のトップページは全関数表をやめてダッシュボードに切り替わります（進捗サマリ・要対応・あなたへの質問・次の一手・レガシーファイル別の進捗表）。全関数の明細はファイル別のサブページに分かれ、リンクで行き来できます。2000 関数でも「いま何が問題で、次に何をやるか」はトップの 1 画面で分かります。

bigproj リバースエンジニアリング WBS ドメイン知識 規約 ⑥完了検証 ⑦分析 新コード詳細(API)

bigproj リバースエンジニアリング WBS

公開
2026年8月3日

進捗サマリ

関数数	①spec	②test-spec	③test-code	④impl	⑤test
300	0/300	0/300	0/300	0/300	0/300

コールグラフ

関数 300 件のため図は省略（依存は下表の「依存」列を参照）。

Open ISSUES（要人間判断）

なし

次の一手（推奨着手順 上位10）

残り 290 件は関数一覧を参照

#	関数	次フェーズ
1	sub0001	①spec
2	sub0002	①spec
3	sub0003	①spec
4	sub0004	①spec
5	sub0005	①spec
6	sub0006	①spec
7	sub0007	①spec
8	sub0008	①spec
9	sub0009	①spec
10	sub0010	①spec

関数一覧（300 件・レガシーファイル別）

レガシーファイル	関数数	①spec	②test-spec	③test-code	④impl	⑤test	要対応
legacy/mod01.f	30	0/30	0/30	0/30	0/30	0/30	—
legacy/mod02.f	30	0/30	0/30	0/30	0/30	0/30	—
legacy/mod03.f	30	0/30	0/30	0/30	0/30	0/30	—

図 5: 大規模モードの WBS（300 関数の例）。トップは進捗サマリ・次の一手・ファイル別進捗のダッシュボードになり、全関数の明細はファイル別サブページへ分割される

中断しても 1 からやり直しにならない

進捗の正はすべてファイル（functions.json・ledger.json・成果物のフロントマター）にあり、AI の会話コンテキストには置きません。セッションが切れても、日をまたいても:

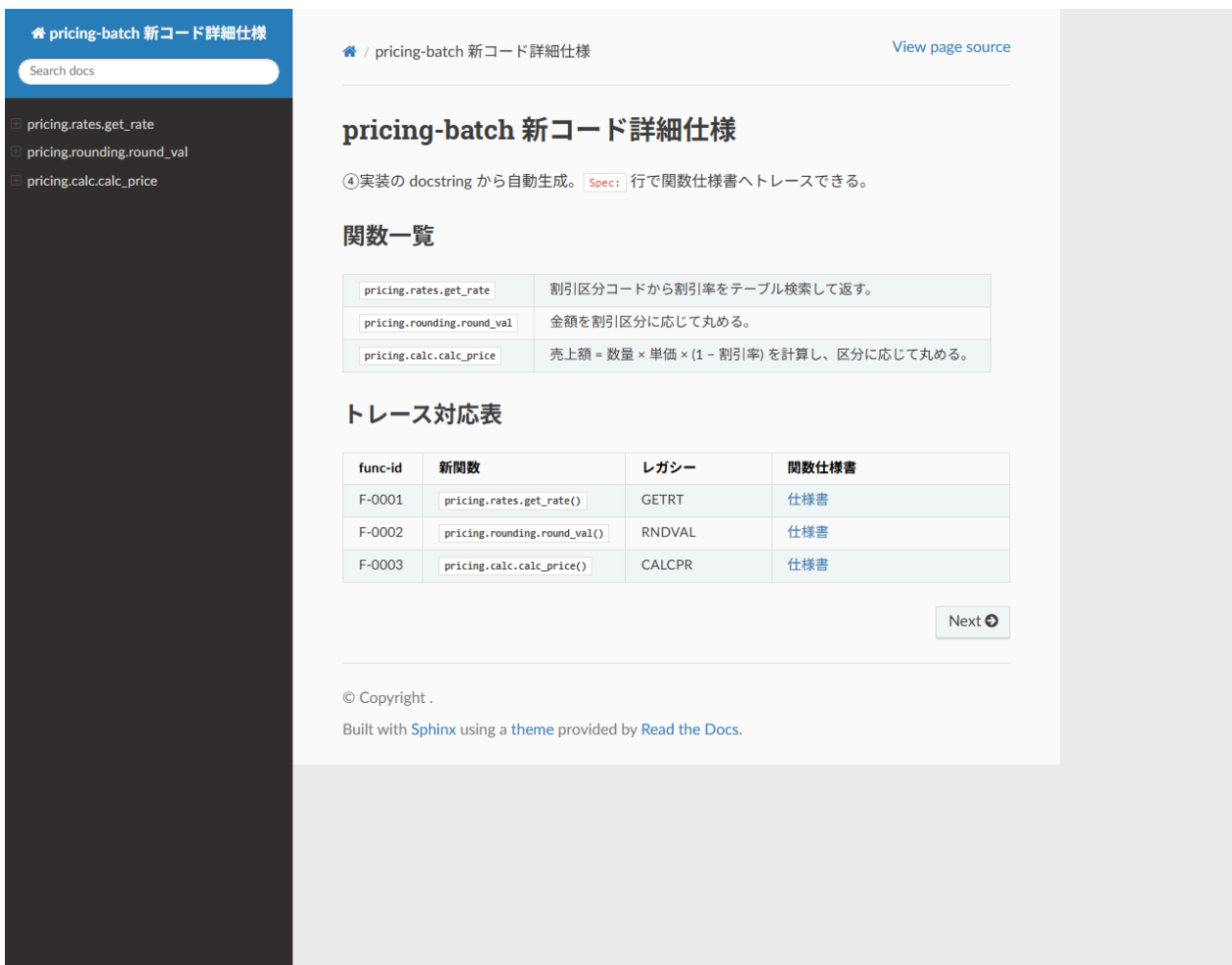
- **再開はいつでも /legacy-reverse を打つだけ**。要約（数行）と着手可能リストを 読んで次の一手を提案してきます。全関数の再列挙はしません
- ①の解析を途中でやり直しても、抽出器は**マージ動作**（関数 ID は不変・手修正は保持・新規だけ追加）なので、採番のやり直しや成果物の宙吊りは起きません
- 「どこまで進んだか分からなくなった」ときは WBS を開くのが最短です。🔴・⚠️・❌ が「要対応」
として最上部にまとまっています

WBS サイト

進捗はすべて HTML サイトで確認します（各フェーズの最後に自動更新されます）。

```
python -m http.server 8765 --directory docs/_site
```

- **トップ = WBS:** 進捗サマリ、コールグラフ図（緑=完了/黄=着手中/灰=未着手/赤=判断待ち。ノードをクリックで仕様書へ）、Open ISSUE（あなたへの質問が常に最上部）、関数一覧（各  から成果物へリンク）、⑦改善イタレーションの達成状況
- ナビバー: ドメイン知識 / 規約 / ⑥完了検証 / ⑦分析 / 新コード詳細(API)
- API ページ (Sphinx) : 関数一覧 → 個別ページ。Spec: 行で仕様書へ、トレース対応表で func-id ↔ 新関数 ↔ レガシー名 ↔ 仕様書が相互に辿れる



pricing-batch 新コード詳細仕様

Search docs

- pricing.rates.get_rate
- pricing.rounding.round_val
- pricing.calc.calc_price

pricing-batch 新コード詳細仕様

④実装の docstring から自動生成。Spec: 行で関数仕様書へトレースできる。

関数一覧

pricing.rates.get_rate	割引区分コードから割引率をテーブル検索して返す。
pricing.rounding.round_val	金額を割引区分に応じて丸める。
pricing.calc.calc_price	売上額 = 数量 × 単価 × (1 - 割引率) を計算し、区分に応じて丸める。

トレース対応表

func-id	新関数	レガシー	関数仕様書
F-0001	pricing.rates.get_rate()	GETRT	仕様書
F-0002	pricing.rounding.round_val()	RNDVAL	仕様書
F-0003	pricing.calc.calc_price()	CALCPR	仕様書

Next

© Copyright .
Built with Sphinx using a theme provided by Read the Docs.

図 6: 新コード詳細(API)。docstring から自動生成され、関数一覧→個別ページ、トレース対応表から仕様書へ渡れる

あなたが直接書いてよいファイル

Table 4

ファイル	手編集	説明
<code>docs/domain-knowledge.md</code> • <code>docs/conventions.md</code>	○	あなたが著者。業務知識・規約はここへ
ISSUE の「回答（人が記入）」欄	○	じっくり書きたい裁定はファイルで
<code>docs/specs/</code> （仕様書）	△	編集可。ハッシュ連鎖が下流を stale にして再確認が走る（設計どおり）
WBS・テスト結果・完了検証レポート	✕	自動生成。次の再生成で消える

ファイルに直接書いた内容は、次にどのフェーズを起動したときに AI が自動で拾います（全 skill は起動時に「回答済み ISSUE・規約変更」をスキャンしてから本処理に入ります）。チャットで「DK に追加して: ○○」と言うだけでも構いません。

⑦ 分析・改善の回し方

⑥が pass した後、`/legacy-7-analyze` で開始します。1 施策 = 1 票 = 1 イタレーションの DevOps ループです。

1. **計測・走査**: AI が `profile / radon / bandit` を実行し、候補を `analysis.md` に列挙。このとき**代表ワークロード (実データ規模の入力) を聞かれるので用意する**。単体テスト負荷だけではホットスポットを断定しません
2. **項目確定**: 着手する候補が施策票 (`docs/improvements/OPT-001.md` など) に昇格。票には**目的・検証可能な成功基準 (baseline→目標)・改善仕様**が書かれる。**あなたはこの票を承認する**
3. **イタレーション**: AI が適用 (1 施策=1 コミット) → テスト全 pass 確認 → 再計測 → 票に実測と判定を記録。全基準達成なら achieved 、未達なら巻き戻して振り返り
4. WBS の「⑦改善イタレーション」表で全施策の達成状況 (//—) がトレースできる

Rust(PyO3) 化のような大きな施策も、AI が 4 段階基準 (CPU バウンドか → NumPy で足りないか → 純粋関数か → 保守コスト明記) で根拠付きの提案を出すので、票の承認で判断してください。

get_rate のフォールバック読込を分離し複雑度を下げる

目的 (Plan)

`get_rate` はプロジェクト唯一の複雑度B評価 (CC=8)。マスタ読込と検索という2つの責務が同居しているため、将来マスタ形式が変わったときの修正範囲を読込側に閉じ込められるよう、読込を `private` 関数 `_load_rate_master()` に分離して両関数をA評価にする。

成功基準 (検証可能な形で定義する)

#	指標	baseline	目標	実測(after)	判定
1	radon CC: <code>get_rate</code>	B(8)	A(5)以下	A(5)	
2	radon CC: 分離後の全関数	—	全てA	<code>get_rate</code> A(5), <code>_load_rate_master</code> A(5)	
3	radon MI: <code>rates.py</code>	A(83.47)	A維持	A(80.02)	
4	⑤テスト (15ケース)	全pass	全pass維持	15 passed	

改善仕様 (Do の設計)

- `rates.py` に `_load_rate_master() -> int` を新設 (0=成功/読込済み扱い、2=ファイル無し)。RATES.DAT のオープン・(`A1,F5.3`) パース・10件打ち切り・STATE への格納をここへ移す
- `get_rate` は「未ロードなら `_load_rate_master()`」、エラー2なら (0,0,2)、あとは検索」に縮む
- **挙動保存の根拠**: 外部から見た入出力 (引数・戻り値・STATE の事後状態・ファイルアクセス) は一切変えない。関数境界の移動のみ。テスト15ケース (②F0001-TC-001~006を含む) が判定する
- `_` 付き `private` のため Sphinx の公開APIには現れない (ドキュメント影響なし)

検証記録 (Check)

- 2026-07-25 適用。 `_load_rate_master()` を新設し読込処理を移動、`get_rate` は判定+検索のみに
- `pytest tests -q` → **15 passed** (②由来の特性テスト全維持。挙動保存を確認)
- `radon cc`: `get_rate` B(8) → **A(5)**、`_load_rate_master` A(5)。B評価消滅
- `radon mi`: A(83.47) → A(80.02) (Aランク維持)

振り返り (Act)

目次

目的 (Plan)

成功基準 (検証可能な形で定義する)

改善仕様 (Do の設計)

検証記録 (Check)

振り返り (Act)

図 7: ⑦施策票の例 (REF-001)。目的→成功基準 (baseline→目標→実測→判定) →検証記録→振り返りが 1 枚に揃い、
目的が達成されたかが後から機械的に追える

Table 5

症状	意味	対処
WBS に  ISSUE-xxx	④⑤ループが上限到達、裁定待ち	ISSUE に回答 → unblock → ⑤再トリガ
WBS に stale△	上流 (①) が改訂され②が古い	/legacy-2-testspec で再生成 → 再承認
WBS に △改変	freeze 後にテストコードが変わった	意図した変更なら③で再 freeze。心当たりがなければ調査
tests/ 編集が拒否された	hook が正常動作している	テスト側の疑義は ISSUE 経由で (正規経路)
再レンダリングが失敗する	配信サーバが _site をロック	サーバの cwd を _site の外にして --directory 指定
図がソースのまま表示される	<code>quarto render docs</code> を直接叩いた	<code>render_site.py</code> で出し直す (Mermaid は影コピー経由でのみ描画される)
WBS の関数名が何行にも折れる	<code>docs/wbs.css</code> が無い / <code>index.qmd</code> を手編集した	<code>ledger wbs</code> で作り直す。CSS はテンプレ (assets/templates/wbs.css) から docs/ にコピー
PDF だけ図が出ない	Mermaid の画像化に Chromium 系ブラウザが要る	<code>qtpdf.py doctor</code> で確認。HTML 側はブラウザが描くので影響なし
⑥で「完全性突合 NG」	抽出器の 2 系統カウントが不一致 (変則的なソース)	<code>extract-report.json</code> の該当ファイルを AI に調査させる。判断がつかなければ ISSUE
①②の機械レビュー NG が報告された	ハルシネーション・省略を機械が検知 (正常動作)	AI が自分で直してから再提出してくる。NG 付きの成果物を承認しない
WBS のファイル別ページが出ない	旧テンプレの <code>_quarto.yml</code> に <code>wbs/*.qmd</code> が無い	<code>render_site.py</code> が自動追記するのでそのまま再実行。恒久対応はテンプレから <code>_quarto.yml</code> を差し替え

PDF 出力

種別ごとの合本 PDF は次で生成します（フォールバック含め自動処理）。

```
python <LR>/scripts/pdf_book.py specs      --root . --output pdf/関数仕様書.pdf      --title  
関数仕様書  
python <LR>/scripts/pdf_book.py test-specs  --root . --output pdf/テスト仕様書.pdf    --title  
テスト仕様書  
python <LR>/scripts/pdf_book.py test-results --root . --output pdf/テスト結果報告書.pdf --  
title テスト結果報告書
```

生成後は `qtpdf.py check <pdf>` で豆腐・はみ出しの機械チェックができます。

付録: 成果物とデータの全体像

docs/	
index.qmd	WBS (自動生成・手編集禁止)
wbs/	200 関数超で自動生成されるファイル別明細ページ (手編集禁止)
conventions.md	プロジェクト規約 (⑩で確定・人が編集可)
domain-knowledge.md	裁定の蓄積 (人が編集可)
specs/F-xxxx.md	① 関数仕様書 (skeleton→draft→reviewed)
test-specs/F-xxxx.md	② テスト仕様書 (generated→approved)
test-results/F-xxxx_日時.md	⑤ 結果報告書 (毎回新規・自動生成)
issues/ISSUE-xxx.md	質問と裁定 (回答欄はあなたが記入可)
improvements/XXX-nnn.md	⑦ 施策票 (proposed→approved→applied→verified)
completion-check.md	⑥ 完了検証 (自動生成)
data/	
functions.json	⑩の解析結果 (正データ。Fortran は機械抽出が生成・マージ)
extract-report.json	⑩機械抽出の監査ログ (完全性突合・推定呼出・未解決名)
ledger.json	ハッシュ・ブロック状態 (スクリプト専用)
src/ , tests/	④実装 / ③テスト (④⑤中は hook が tests/ を保護)

ステータスの意味: =完了 / =作業中 (draft 等) / =未着手 / =裁定待ち / =要再確認